Министерство образования Республики Беларусь

Учреждения образования

«Брестский государственный технический университет

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1

Специальность ПО-12

Выполнил
К. В. Ковалюк,
студент группы ПО-12

Проверил
А. Д. Кулик,
преп. стаж. кафедры ИИТ,
«___» _____2026 г.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python.

Вариант: №7

Задание 1:

Реализовать простой класс

Требования к выполнению:

• Реализовать пользовательский класс по варианту.

Для каждого класса:

• Создать атрибуты (поля) классов

• Создать методы классов

• Добавить необходимые свойства и сеттеры (по необходимости)

• Переопределить магические методы __str__ и __eq__

7) Множество символов ограниченной мощности - Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Выполнение:

Код программы

```
class CharSet:
    def __init__(self, max_size, init_chars=None):
        if max_size <= 0:
            raise ValueError("Мощность множества должна быть положительным числом.")
        self._max_size = max_size
        self._items = []
        if init_chars:
            for char in init_chars:
                self.add(char)

    @property
    def max_size(self):
        return self._max_size

    @property
    def items(self):
        return self._items.copy()

    def add(self, char):
        if not isinstance(char, str) or len(char) != 1:
            print("Ошибка: Можно добавлять только одиночные символы.")
            return False
        if char in self._items:
            print(f"Элемент '{char}' уже существует в множестве.")
            return False
        if len(self._items) >= self._max_size:
            print(f"Ошибка: Достигнута максимальная мощность множества ({self._max_size}). Элемент '{char}' не добавлен.")
            return False
        self._items.append(char)
        print(f"Элемент '{char}' успешно добавлен.")
        return True

    def remove(self, char):
        if char in self._items:
            self._items.remove(char)
            print(f"Элемент '{char}' удален.")
            return True
        else:
            print(f"Элемент '{char}' не найден в множестве.")
```

```
        return False

    def contains(self, char):
        return char in self._items

    def union(self, other_set):
        if not isinstance(other_set, CharSet):
            raise TypeError("Операнд должен быть объектом класса CharSet")

        new_max_size = self.max_size + other_set.max_size
        combined_chars = list(self._items)
        for char in other_set.items:
            if char not in combined_chars:
                combined_chars.append(char)

        new_set = CharSet(new_max_size)
        for char in combined_chars:
            new_set.add(char)
        return new_set

    def display(self):
        print(f"Множество (мощность {len(self._items)}/{self._max_size}): {self._items}")

    def __str__(self):
        return f"CharSet(capacity={self._max_size}, items={self._items})"

    def __eq__(self, other):
        if not isinstance(other, CharSet):
            return False
        return (self._max_size == other._max_size and
                set(self._items) == set(other._items))


if __name__ == "__main__":
    print("--- Демонстрация работы класса CharSet ---")

    set1 = CharSet(5, "abracadabra")
    print("Создано set1 из строки 'abracadabra' (макс. 5):")
    set1.display()

    set2 = CharSet(3, "xyz")
    print("\nСоздано set2 из строки 'xyz' (макс. 3):")
    set2.display()

    print("\n--- Операции добавления/удаления ---")
    set1.add('c')
    set1.add('d')
    set1.add('e')
    set1.add('f')
    set1.display()

    set1.remove('a')
    set1.display()
    set1.remove('z')

    print("\n--- Проверка принадлежности ---")
    print(f"Содержит ли set1 символ 'b'? {set1.contains('b')}")
    print(f"Содержит ли set1 символ 'z'? {set1.contains('z')}")

    print("\n--- Объединение множеств ---")
    set3 = set1.union(set2)
    print("Результат объединения set1 и set2:")
    set3.display()
    print(f"Строковое представление set3: {set3}")

    print("\n--- Сравнение множеств ---")
```

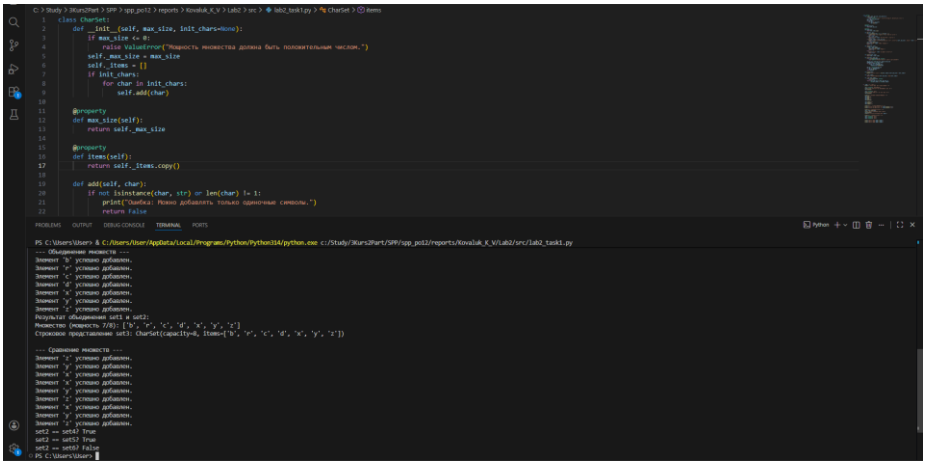
```
set4 = CharSet(3, "zyx")
set5 = CharSet(3, "xyz")
set6 = CharSet(4, "xyz")

print(f"set2 == set4? {set2 == set4}")
print(f"set2 == set5? {set2 == set5}")
print(f"set2 == set6? {set2 == set6}")
```

Результат работы программы:

```
Элемент 'a' успешно добавлен.
Элемент 'b' успешно добавлен.
Элемент 'r' успешно добавлен.
Элемент 'a' уже существует в множестве.
Элемент 'c' успешно добавлен.
Элемент 'a' уже существует в множестве.
Элемент 'd' успешно добавлен.
Элемент 'a' уже существует в множестве.
Элемент 'b' уже существует в множестве.
Элемент 'r' уже существует в множестве.
Элемент 'a' уже существует в множестве.
Создано set1 из строки 'abracadabra' (макс. 5):
Множество (мощность 5/5): ['a', 'b', 'r', 'c', 'd']
Элемент 'x' успешно добавлен.
Элемент 'y' успешно добавлен.
Элемент 'z' успешно добавлен.

Создано set2 из строки 'xyz' (макс. 3):
Множество (мощность 3/3): ['x', 'y', 'z']

--- Операции добавления/удаления ---
Элемент 'c' уже существует в множестве.
Элемент 'd' уже существует в множестве.
Ошибка: Достигнута максимальная мощность множества (5). Элемент 'e' не добавлен.
Ошибка: Достигнута максимальная мощность множества (5). Элемент 'f' не добавлен.
Множество (мощность 5/5): ['a', 'b', 'r', 'c', 'd']
Элемент 'a' удален.
Множество (мощность 4/5): ['b', 'r', 'c', 'd']
Элемент 'z' не найден в множестве.

--- Проверка принадлежности ---
Содержит ли set1 символ 'b'? True
Содержит ли set1 символ 'z'? False

--- Объединение множеств ---
Элемент 'b' успешно добавлен.
Элемент 'r' успешно добавлен.
Элемент 'c' успешно добавлен.
Элемент 'd' успешно добавлен.
Элемент 'x' успешно добавлен.
Элемент 'y' успешно добавлен.
Элемент 'z' успешно добавлен.
Результат объединения set1 и set2:
Множество (мощность 7/8): ['b', 'r', 'c', 'd', 'x', 'y', 'z']
Строковое представление set3: CharSet(capacity=8, items=['b', 'r', 'c', 'd', 'x', 'y', 'z'])

--- Сравнение множеств ---
Элемент 'z' успешно добавлен.
Элемент 'y' успешно добавлен.
Элемент 'x' успешно добавлен.
Элемент 'x' успешно добавлен.
Элемент 'y' успешно добавлен.
Элемент 'z' успешно добавлен.
Элемент 'x' успешно добавлен.
Элемент 'y' успешно добавлен.
Элемент 'z' успешно добавлен.
set2 == set4? True
```

set2 == set5? True
set2 == set6? False

Рисунок с результатом работы программы:



Задание 2:

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы

Система Автобаза. Диспетчер распределяет заявки на Рейсы между Водителями и назначает для этого Автомобиль. Водитель может сделать заявку на ремонт. Диспетчер может отстранить Водителя от работы. Водитель делает отметку о выполнении Рейса и состоянии Автомобиля.

Выполнение:

Код программы

```
from enum import Enum
from datetime import datetime

class CarStatus(Enum):
    OK = "Исправен"
    BROKEN = "Требует ремонта"
    IN_REPAIR = "В ремонте"

class TripStatus(Enum):
    PENDING = "Ожидает назначения"
    ASSIGNED = "Назначен"
    COMPLETED = "Выполнен"
    CANCELLED = "Отменен"

class Person:
    def __init__(self, name, person_id):
        self.name = name
        self.person_id = person_id

    def __str__(self):
        return f"{self.name} (ID: {self.person_id})"

class Driver(Person):
    def __init__(self, name, person_id, experience_years):
        super().__init__(name, person_id)
        self.experience_years = experience_years
        self.is_active = True
        self.assigned_car = None
        self.trip_history = []

    def request_repair(self):
        if self.assigned_car and self.assigned_car.status == CarStatus.BROKEN:
```

```
            print(f"Водитель {self.name} заявляет о поломке автомобиля {self.assigned_car.model}.")
            return True
        elif self.assigned_car:
            print(f"Водитель {self.name}: автомобиль {self.assigned_car.model} исправен, ремонт не требуется.")
            return False
        else:
            print(f"Водитель {self.name}: за мной не закреплен автомобиль.")
            return False

    def complete_trip(self, trip, car_condition_ok=True):
        if trip.assigned_driver == self and trip.status == TripStatus.ASSIGNED:
            trip.status = TripStatus.COMPLETED
            self.trip_history.append(trip)
            print(f"Водитель {self.name} выполнил рейс {trip.trip_id}.")

            if not car_condition_ok and self.assigned_car:
                self.assigned_car.status = CarStatus.BROKEN
                print(f"Водитель {self.name} отметил, что автомобиль {self.assigned_car.model} неисправен.")
            return True
        else:
            print(f"Ошибка: Рейс {trip.trip_id} не может быть завершен водителем {self.name}.")
            return False

    def __str__(self):
        status = "активен" if self.is_active else "отстранен"
        car = self.assigned_car.model if self.assigned_car else "нет авто"
        return f"Водитель {self.name}, стаж: {self.experience_years}, {status}, авто: {car}"

class Car:
    def __init__(self, model, license_plate):
        self.model = model
        self.license_plate = license_plate
        self.status = CarStatus.OK

    def __str__(self):
        return f"Авто {self.model} ({self.license_plate}), статус: {self.status.value}"

class Trip:
    def __init__(self, trip_id, destination, distance_km):
        self.trip_id = trip_id
        self.destination = destination
        self.distance_km = distance_km
        self.status = TripStatus.PENDING
        self.assigned_driver = None
        self.assigned_car = None

    def __str__(self):
        driver_name = self.assigned_driver.name if self.assigned_driver else "не назначен"
        return f"Рейс {self.trip_id} до {self.destination}, статус: {self.status.value}, водитель: {driver_name}"

class Dispatcher(Person):
    def __init__(self, name, person_id):
        super().__init__(name, person_id)
        self._drivers = []
        self._cars = []
        self._trips = []

    def add_driver(self, driver):
        self._drivers.append(driver)
        print(f"Диспетчер {self.name}: Водитель {driver.name} добавлен в систему.")

    def add_car(self, car):
        self._cars.append(car)
        print(f"Диспетчер {self.name}: Автомобиль {car.model} добавлен в автопарк.")
```

```
    def add_trip_request(self, trip):
        self._trips.append(trip)
        print(f"Диспетчер {self.name}: Поступила заявка на рейс {trip.trip_id} до {trip.destination}.")

    def assign_trip(self, trip_id, driver_id, car_license):
        trip = next((t for t in self._trips if t.trip_id == trip_id), None)
        driver = next((d for d in self._drivers if d.person_id == driver_id and d.is_active), None)
        car = next((c for c in self._cars if c.license_plate == car_license and c.status == CarStatus.OK), None)

        if not trip or trip.status != TripStatus.PENDING:
            print(f"Ошибка: Рейс {trip_id} не найден или не ожидает назначения.")
            return False
        if not driver:
            print(f"Ошибка: Водитель с ID {driver_id} не найден, неактивен или отстранен.")
            return False
        if not car:
            print(f"Ошибка: Автомобиль {car_license} не найден или неисправен.")
            return False

        trip.status = TripStatus.ASSIGNED
        trip.assigned_driver = driver
        trip.assigned_car = car
        driver.assigned_car = car
        print(f"Диспетчер {self.name}: Рейс {trip_id} назначен водителю {driver.name} на автомобиле
{car.model}.")
        return True

    def suspend_driver(self, driver_id):
        driver = next((d for d in self._drivers if d.person_id == driver_id), None)
        if driver:
            driver.is_active = False
            print(f"Диспетчер {self.name}: Водитель {driver.name} отстранен от работы.")
            return True
        else:
            print(f"Ошибка: Водитель с ID {driver_id} не найден.")
            return False

    def process_repair_request(self, driver_id):
        driver = next((d for d in self._drivers if d.person_id == driver_id), None)
        if driver and driver.assigned_car and driver.assigned_car.status == CarStatus.BROKEN:
            car = driver.assigned_car
            car.status = CarStatus.IN_REPAIR
            print(f"Диспетчер {self.name}: Принята заявка от {driver.name}. Автомобиль {car.model}
отправлен в ремонт.")
            car.status = CarStatus.OK
            print(f"Диспетчер {self.name}: Автомобиль {car.model} отремонтирован и снова готов к работе.")
            return True
        else:
            print(f"Диспетчер {self.name}: Заявка на ремонт от {driver.name} не может быть обработана.")
            return False

    def show_system_state(self):
        print("\n" + "="*50)
        print(f"СОСТОЯНИЕ СИСТЕМЫ (Диспетчер {self.name})")
        print("-"*50)
        print("Водители:")
        for d in self._drivers:
            print(f"  {d}")
        print("Автомобили:")
        for c in self._cars:
            print(f"  {c}")
        print("Рейсы:")
        for t in self._trips:
            print(f"  {t}")
        print("="*50 + "\n")
```

```
if __name__ == "__main__":
    print("--- Демонстрация работы системы 'Автобаза' ---")

    disp = Dispatcher("Анна Петровна", "D001")

    d1 = Driver("Иван Сидоров", "DR001", 5)
    d2 = Driver("Петр Иванов", "DR002", 2)
    d3 = Driver("Сергей Смирнов", "DR003", 10)

    c1 = Car("ГАЗель Next", "A123BB")
    c2 = Car("Ford Transit", "B456AK")
    c3 = Car("Volvo FH", "E789KM")

    print("\n--- Инициализация системы ---")
    disp.add_driver(d1)
    disp.add_driver(d2)
    disp.add_driver(d3)
    disp.add_car(c1)
    disp.add_car(c2)
    disp.add_car(c3)

    print("\n--- Поступление заявок ---")
    t1 = Trip("T101", "Минск", 120)
    t2 = Trip("T102", "Брест", 350)
    t3 = Trip("T103", "Гродно", 280)
    disp.add_trip_request(t1)
    disp.add_trip_request(t2)
    disp.add_trip_request(t3)

    disp.show_system_state()

    print("\n--- Распределение рейсов ---")
    disp.assign_trip("T101", "DR001", "A123BB")
    disp.assign_trip("T102", "DR002", "B456AK")
    disp.assign_trip("T103", "DR003", "НЕТНОМЕР")

    disp.show_system_state()

    print("\n--- Выполнение рейсов и события ---")
    d1.complete_trip(t1, car_condition_ok=False)

    d1.request_repair()

    disp.process_repair_request("DR001")

    disp.suspend_driver("DR002")

    d2.complete_trip(t2)

    disp.show_system_state()
```

Результат работы программы:

```
--- Демонстрация работы системы 'Автобаза' ---

--- Инициализация системы ---
Диспетчер Анна Петровна: Водитель Иван Сидоров добавлен в систему.
Диспетчер Анна Петровна: Водитель Петр Иванов добавлен в систему.
Диспетчер Анна Петровна: Водитель Сергей Смирнов добавлен в систему.
Диспетчер Анна Петровна: Автомобиль ГАЗель Next добавлен в автопарк.
Диспетчер Анна Петровна: Автомобиль Ford Transit добавлен в автопарк.
Диспетчер Анна Петровна: Автомобиль Volvo FH добавлен в автопарк.

--- Поступление заявок ---
Диспетчер Анна Петровна: Поступила заявка на рейс T101 до Минск.
Диспетчер Анна Петровна: Поступила заявка на рейс T102 до Брест.
```

Диспетчер Анна Петровна: Поступила заявка на рейс T103 до Гродно.

==========
СОСТОЯНИЕ СИСТЕМЫ (Диспетчер Анна Петровна)

Водители:
Водитель Иван Сидоров, стаж: 5, активен, авто: нет авто
Водитель Петр Иванов, стаж: 2, активен, авто: нет авто
Водитель Сергей Смирнов, стаж: 10, активен, авто: нет авто
Автомобили:
Авто ГАЗель Next (A123BB), статус: Исправен
Авто Ford Transit (B456AK), статус: Исправен
Авто Volvo FH (E789KM), статус: Исправен
Рейсы:
Рейс T101 до Минск, статус: Ожидает назначения, водитель: не назначен
Рейс T102 до Брест, статус: Ожидает назначения, водитель: не назначен
Рейс T103 до Гродно, статус: Ожидает назначения, водитель: не назначен
==========

--- Распределение рейсов ---
Диспетчер Анна Петровна: Рейс T101 назначен водителю Иван Сидоров на автомобиле ГАЗель Next.
Диспетчер Анна Петровна: Рейс T102 назначен водителю Петр Иванов на автомобиле Ford Transit.
Ошибка: Автомобиль НЕТНОМЕР не найден или неисправен.

==========
СОСТОЯНИЕ СИСТЕМЫ (Диспетчер Анна Петровна)

Водители:
Водитель Иван Сидоров, стаж: 5, активен, авто: ГАЗель Next
Водитель Петр Иванов, стаж: 2, активен, авто: Ford Transit
Водитель Сергей Смирнов, стаж: 10, активен, авто: нет авто
Автомобили:
Авто ГАЗель Next (A123BB), статус: Исправен
Авто Ford Transit (B456AK), статус: Исправен
Авто Volvo FH (E789KM), статус: Исправен
Рейсы:
Рейс T101 до Минск, статус: Назначен, водитель: Иван Сидоров
Рейс T102 до Брест, статус: Назначен, водитель: Петр Иванов
Рейс T103 до Гродно, статус: Ожидает назначения, водитель: не назначен
==========

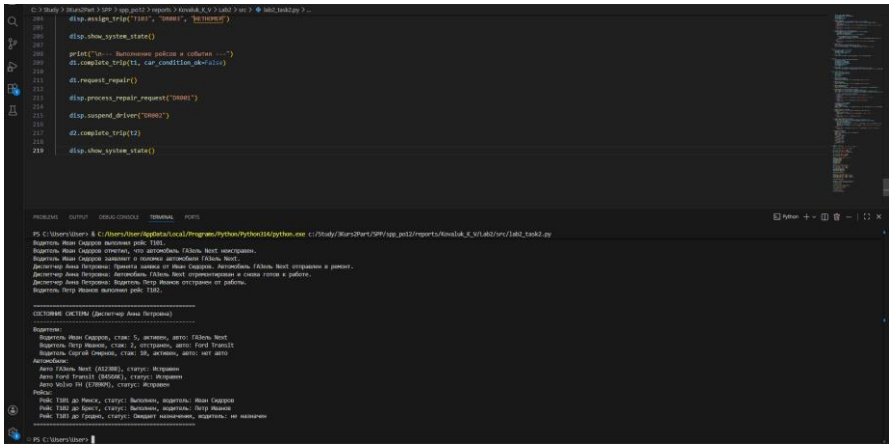
--- Выполнение рейсов и события ---
Водитель Иван Сидоров выполнил рейс T101.
Водитель Иван Сидоров отметил, что автомобиль ГАЗель Next неисправен.
Водитель Иван Сидоров заявляет о поломке автомобиля ГАЗель Next.
Диспетчер Анна Петровна: Принята заявка от Иван Сидоров. Автомобиль ГАЗель Next отправлен в ремонт.
Диспетчер Анна Петровна: Автомобиль ГАЗель Next отремонтирован и снова готов к работе.
Диспетчер Анна Петровна: Водитель Петр Иванов отстранен от работы.
Водитель Петр Иванов выполнил рейс T102.

==========
СОСТОЯНИЕ СИСТЕМЫ (Диспетчер Анна Петровна)

Водители:
Водитель Иван Сидоров, стаж: 5, активен, авто: ГАЗель Next
Водитель Петр Иванов, стаж: 2, отстранен, авто: Ford Transit
Водитель Сергей Смирнов, стаж: 10, активен, авто: нет авто
Автомобили:
Авто ГАЗель Next (A123BB), статус: Исправен
Авто Ford Transit (B456AK), статус: Исправен
Авто Volvo FH (E789KM), статус: Исправен
Рейсы:
Рейс T101 до Минск, статус: Выполнен, водитель: Иван Сидоров

Рейс T102 до Брест, статус: Выполнен, водитель: Петр Иванов
Рейс T103 до Гродно, статус: Ожидает назначения, водитель: не назначен
==========

Рисунок с результатом работы программы:



Вывод: В ходе работы я закрепил базовые знания языка программирования Python при решении практических задач.